

From Attacker to Defender

Cybersecurity Internship 2026 • DevelopersHub • Node.js Security

Week 1 was about breaking things. Week 2 was about making sure no one else could. After identifying 10 vulnerabilities — including SQL Injection, Stored XSS, plain-text passwords, and 7 ZAP-flagged misconfigurations — this week I implemented a full security layer from scratch.

"Most apps are not hacked because attackers are geniuses. They are hacked because developers never tested their own code from the outside."

❖ Security Features Implemented

Fix	Library / Method	What It Solves
Password Hashing	bcryptjs (salt: 10)	Plain-text password exposure
JWT Authentication	jsonwebtoken + HTTP-only cookies	Session hijacking, insecure auth
Input Validation	validator library	XSS & injection via user input
CSRF Protection	csrf + sameSite: strict	Cross-site request forgery
Rate Limiting	express-rate-limit (5/15 min)	Brute-force login attacks
Security Headers	helmet.js (11+ headers)	Clickjacking, MIME sniff, CSP
Tech Stack Hiding	app.disable('x-powered-by')	Server fingerprinting by attacker
SQL Injection Fix	Parameterized queries (?)	Database manipulation / bypass

❖ Key Code Changes

Password Hashing — bcryptjs

```
const bcrypt = require('bcryptjs');
// On signup – hash before saving:
const hash = await bcrypt.hash(password, 10);
db.run('INSERT INTO users VALUES (?,?)', [username, hash]);

// On login – compare safely:
const match = await bcrypt.compare(input, storedHash);
```

SQL Injection Fix — Parameterized Queries

```
// ■ BEFORE (vulnerable):  
db.get(`SELECT * FROM users WHERE username='${u}'`);  
  
// ■ AFTER (safe):  
db.get('SELECT * FROM users WHERE username=?', [u], cb);
```

JWT Authentication + HTTP-only Cookie

```
const jwt = require('jsonwebtoken');  
const token = jwt.sign({ id: user.id }, process.env.JWT_SECRET);  
res.cookie('token', token, { httpOnly: true, sameSite: 'strict' });
```

Helmet.js — Security Headers in One Line

```
const helmet = require('helmet');  
app.use(helmet());  
// Sets: CSP, X-Frame-Options, X-Content-Type-Options,  
// Referrer-Policy, HSTS, and 6 more headers automatically.
```

❖ Before vs After — Full Comparison

Aspect	Week 1 (Vulnerable) ■	Week 2 (Secure) ■
Password storage	Plain text in DB	bcrypt hash (10 rounds)
Input handling	Raw — no filtering	Sanitized & validated
Authentication	None	JWT + HTTP-only cookie
Session security	None	SameSite strict, no JS access
Security headers	None — 7 ZAP alerts	11+ headers via Helmet.js
CSRF protection	No tokens on forms	csrf tokens + strict cookies
Brute-force	Unlimited attempts	5 attempts / 15 minutes
Tech stack leak	X-Powered-By: Express	Hidden — disabled
SQL queries	String concatenation	Parameterized (? placeholders)
ZAP high-risk alerts	10 vulnerabilities	0 high-risk remaining ■

❖ Risk Matrix — Final Status

Vulnerability	Tool	Score	Level	Status
SQL Injection (Auth Bypass)	Manual	25	Critical	■ Patched
Plain-Text Password Storage	Manual	20	Critical	■ Patched
Stored XSS (Profile Bio)	Manual	16	Critical	■ Patched
Absence of Anti-CSRF Tokens	OWASP ZAP	12	High	■ Patched
Missing Anti-clickjacking Header	OWASP ZAP	9	Medium	■ Patched
CSP Header Not Set	OWASP ZAP	8	Medium	■ Patched
CSP: No Fallback Directive	OWASP ZAP	8	Medium	■ Patched
X-Powered-By Leaks Tech Stack	OWASP ZAP	5	Low	■ Patched
X-Content-Type-Options Missing	OWASP ZAP	4	Low	■ Patched
Authentication Request Exposed	OWASP ZAP	2	Info	■ Noted

Final Audit Result

0 High-Risk Vulnerabilities Remaining Verified by OWASP ZAP Post-Remediation Scan

❖ Project Structure

File / Folder	Purpose
app.js	Main Express server — secured routes
middleware/auth.js	JWT authentication middleware
views/login.ejs	Login form with CSRF token protection
views/signup.ejs	Signup with validation hints
views/profile.ejs	Protected user dashboard
ScreenShots/	Evidence from vulnerability testing
ZAP_Report_Week1.html	OWASP ZAP scan — before remediation
ZAP_Report_Week2.html	OWASP ZAP scan — after remediation
Cybersecurity_Report_Week1_Final.pdf	Formal Week 1 findings document

GitHub: github.com/WahabShahbaz551/Cyber-Security-Internship-2026 • Week 3 coming soon: Logging, Nmap Pen Testing & Final Report